

#	Area	Learn	Concrete project	If you can build it, explain why you designed it that way, measure it, break it, and fix it—you probably understand it.	Move on when...
1	Python	Syntax → functions → OOP → modules → typing → exceptions → testing → async basics	CLI application, then turn it into a small service	You can write/debug ~1,000 lines without copying tutorials	You can build normal applications independently
2	HTTP & APIs	HTTP methods, status codes, headers, JSON, REST, authentication, cookies, JWT, WebSockets	Build a URL shortener API	You can explain every request from client → server → DB → response	You can design an API from requirements
3	SQL & PostgreSQL	Schema design, joins, indexes, transactions, isolation, constraints, query plans	Add PostgreSQL to URL shortener	Create indexes based on actual queries; demonstrate transaction/concurrency problems	You can design a production database schema
4	Backend Engineering	Service architecture, validation, error handling, caching, background jobs, testing, logging	Production URL shortener with users, analytics, Redis, PostgreSQL	Tests + authentication + caching + async jobs + proper failure handling	You can build a backend without a tutorial
5	Linux + Networking	Processes, memory, ports, DNS, TCP, TLS, reverse proxies, load balancing basics	Deploy backend to a Linux server and debug it	Break DNS/network/processes and diagnose the problem yourself	You understand what happens below your application
6	Docker	Images, containers, volumes, networking, Compose	Containerize API + DB + Redis	Clone repo on another machine and start everything reproducibly	You understand container isolation/networking
7	Cloud basics	Compute, networking, IAM, storage, managed DBs, secrets, monitoring	Deploy your backend to AWS/Azure/GCP	Secure deployment with HTTPS, private DB and logs	You can deploy/troubleshoot without relying on a tutorial
8	System Design Fundamentals	Scalability, availability, latency, throughput, load balancing, caching, queues, DB replication	Redesign URL shortener for 10M users	Calculate traffic/storage and identify bottlenecks	You can defend architecture decisions
9	Distributed Systems	Replication, partitioning, sharding, consistency, idempotency, retries, eventual consistency, Kafka	Build distributed URL analytics	Kill components, duplicate messages, create delays and recover correctly	You understand failure between independent services
10	Advanced System Design	Distributed transactions, rate limiting, CDNs, service discovery, observability, fault tolerance	Design Uber/WhatsApp/YouTube-like system	You can handle ambiguous requirements and tradeoffs	You can design a large system on a whiteboard and explain tradeoffs
11	ML Fundamentals	Regression, classification, features, training/evaluation, overfitting, precision/recall	Fraud detection service	Explain why the model works/fails and deploy it as an API	You understand ML beyond calling a library
12	PyTorch / Deep Learning	Tensors, neural networks, loss, backprop, optimizers, GPU, CNNs	Image classifier in PyTorch	Explain and modify the training loop yourself	You understand what training is actually doing
13	Transformers	Tokens, embeddings, attention, self-attention, transformer blocks	Build a tiny transformer	Implement attention yourself and explain it	You understand transformers rather than just using them
14	LLM Engineering	LLM APIs, context, structured output, tool calling, inference, token/cost management	Build an AI assistant backend	Production-style API with retries, streaming, auth, rate limits	You understand how an LLM fits into a backend system
15	RAG	Embeddings, chunking, vector DB, retrieval, reranking, citations, evaluation	Enterprise document assistant	Measure retrieval quality, hallucination and latency	You can explain and improve retrieval failures
16	AI Agents	Tool use, state, planning, memory, permissions, retries, evaluation	Research/automation agent	Deliberately break tools/sources and safely recover	You understand when agents are useful and their failure modes
17	Production AI	Caching, evaluation, observability, security, cost, latency, model selection	Deploy RAG/agent system	Dashboard for quality, latency, errors, token usage and cost	You optimize based on measurements
18	AI + System Design	AI-specific architecture, model serving, queues, vector DBs, caching, async inference, fallbacks	Design production AI platform for 1M users	Explain architecture, bottlenecks, cost and failure modes	You can design AI systems at scale
19	Capstone	Everything above	AI-powered e-commerce platform	Backend + distributed architecture + LLM + RAG + agents + production deployment	You can build, break, measure and improve the whole system